



Microservice load balancing

A.A. 2025/2026

Service Oriented Software Engineering

Gianluca Filippone

gianluca.filippone@gssi.it

Scalability

Scalability is the property of a system of being able to handle a growing amount of work

Two approaches:

- Vertical scaling** (vary the amount of resource)

- Horizontal scaling** (delegate workload among different nodes)

Horizontal scaling brings multiple benefits: **performances**, **availability**, **fault tolerance**

(In microservices) Realizing horizontal scaling means distribute the incoming requests among different service instances: **Load Balancing**

Load balancing approaches

Load Balancing spans many different areas:

- Distribution of workload across threads/processors/servers

- Network links

- Services

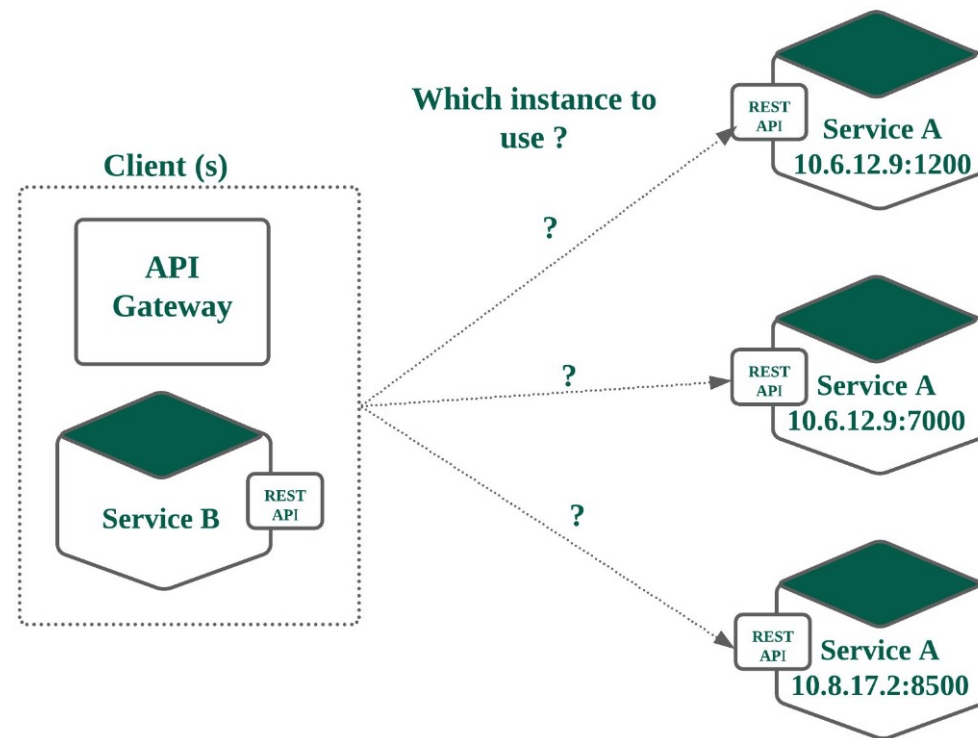
Different approaches for service load balancing

- Server-side load balancing

- Client-side load balancing

- Hybrid approach

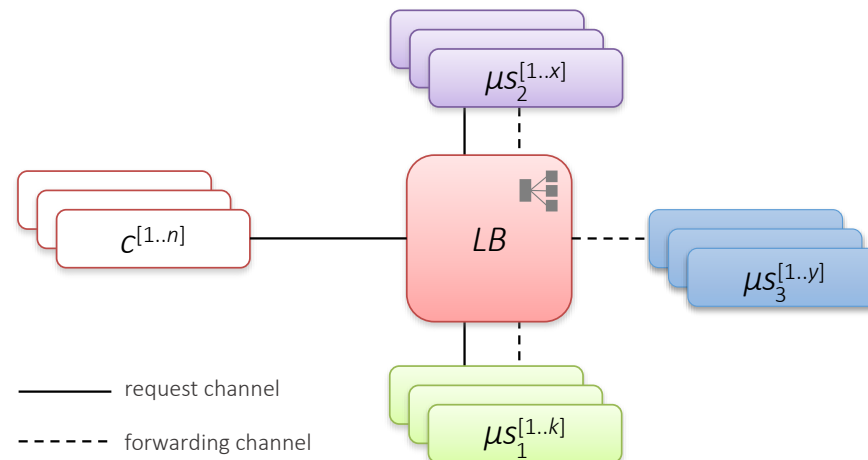
The problem of load balancing



Server-side Load Balancing (1)

Fully-centralized approach: a single load balancer is interposed between the microservices

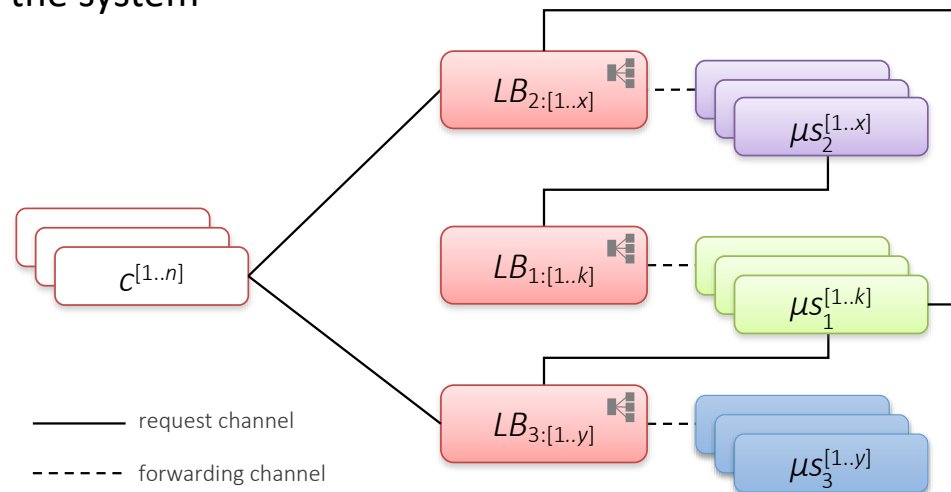
- + Easy to be aware of the workload of each microservice instance
- + Hides the internal structure
- Introduces a single-point of failure
- Bottleneck of the system



Server-side Load Balancing (2)

Partially-centralized approach: a load balancer per microservice type

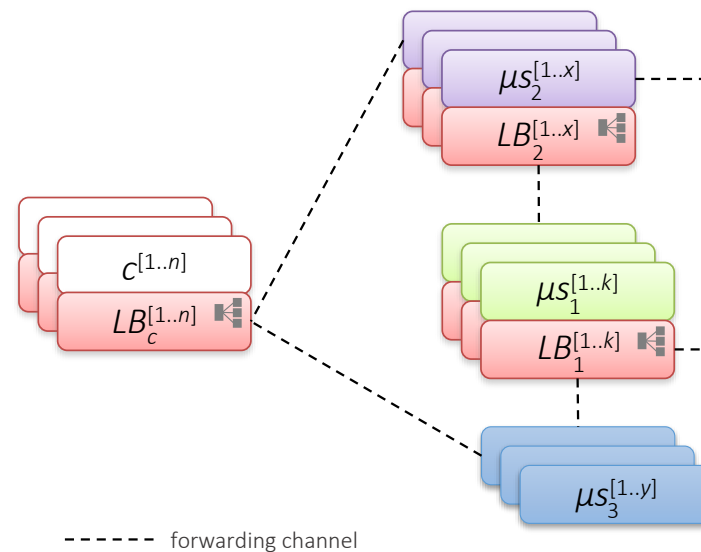
- + Easy to be aware of the workload of each microservice instance
- + Hides the internal structure
- Single-point of failure for each microservice
- Still bottleneck of the system



Client-side Load Balancing

Fully-distributed approach: a local load balancer assigned to each client or consumer instance

- + Improved fault tolerance (no failure points)
- + Bottleneck free
- + No extra-hop due to request re-routing
- + Better scalable approach
- No direct knowledge of instances workload
- No internal structure hiding



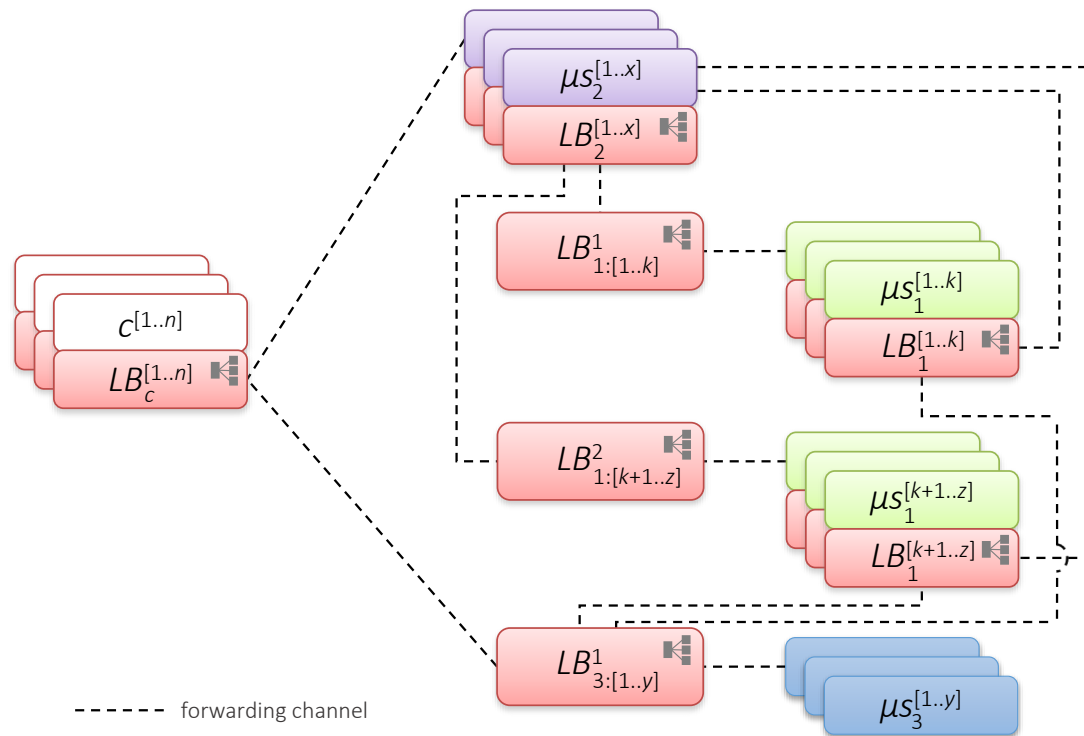
Approaches comparison

	Server-side Load Balancing	Client-side Load Balancing
Security Benefits	✓	✗
No points of failure	✗	✓
No bottlenecks	✗	✓
Automatic scaling	✗	✓
Lighter infrastructure investment	✗	✓
Lower status propagation latency	✓	✗
Higher throughput	✗	✓

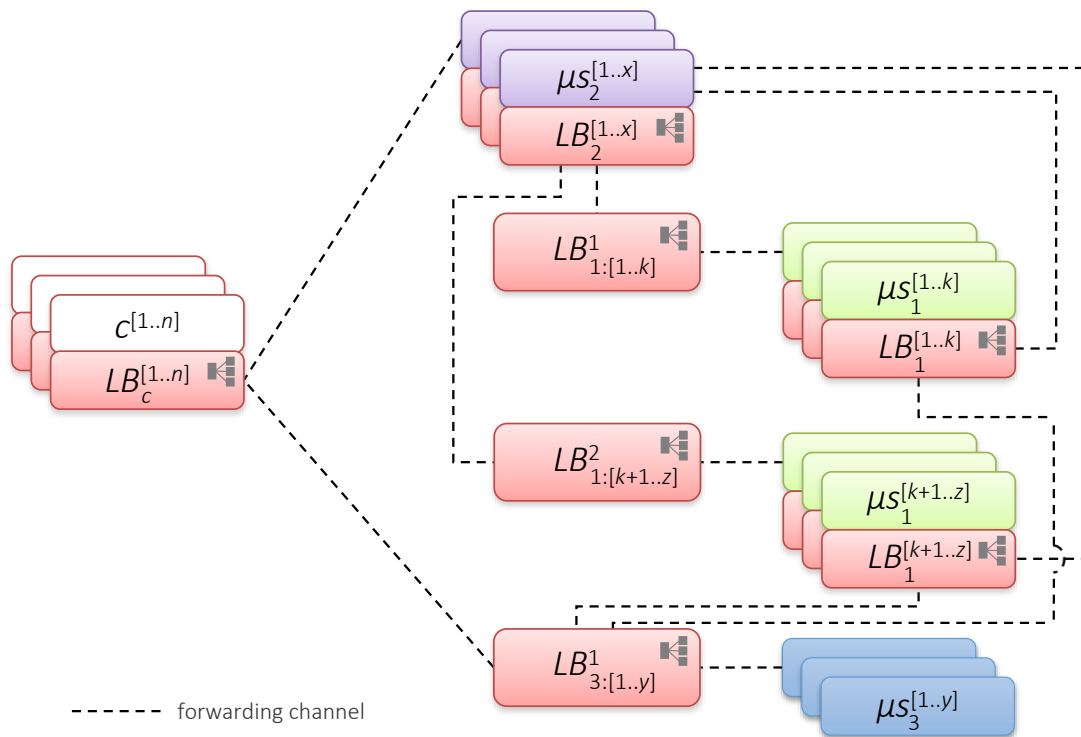
Idea: a **hybrid approach** that brings benefits from both client and server-side load balancing

Hybrid Load Balancing

Hybrid approach: a client-side load balancer can be connected to many instances of a server-side load balancer



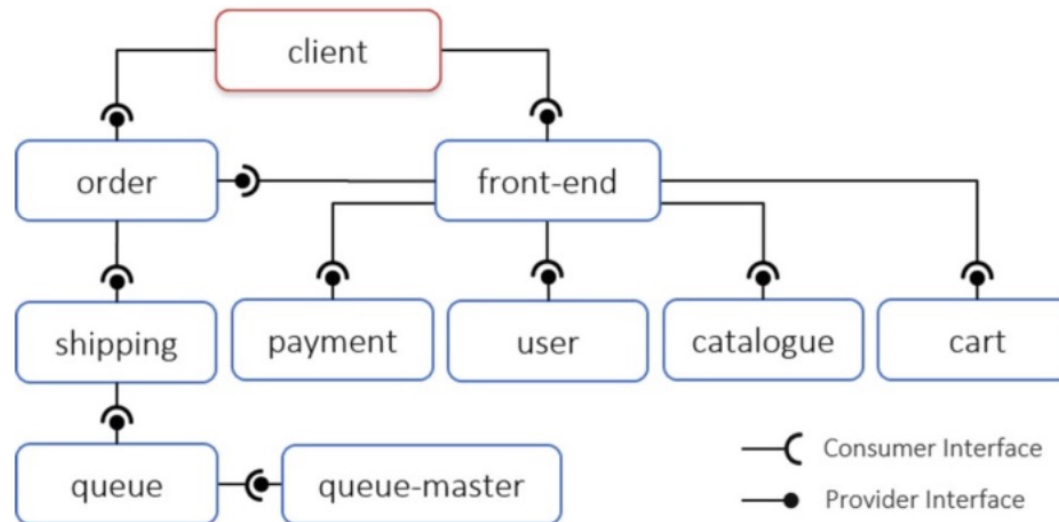
Hybrid Load Balancing



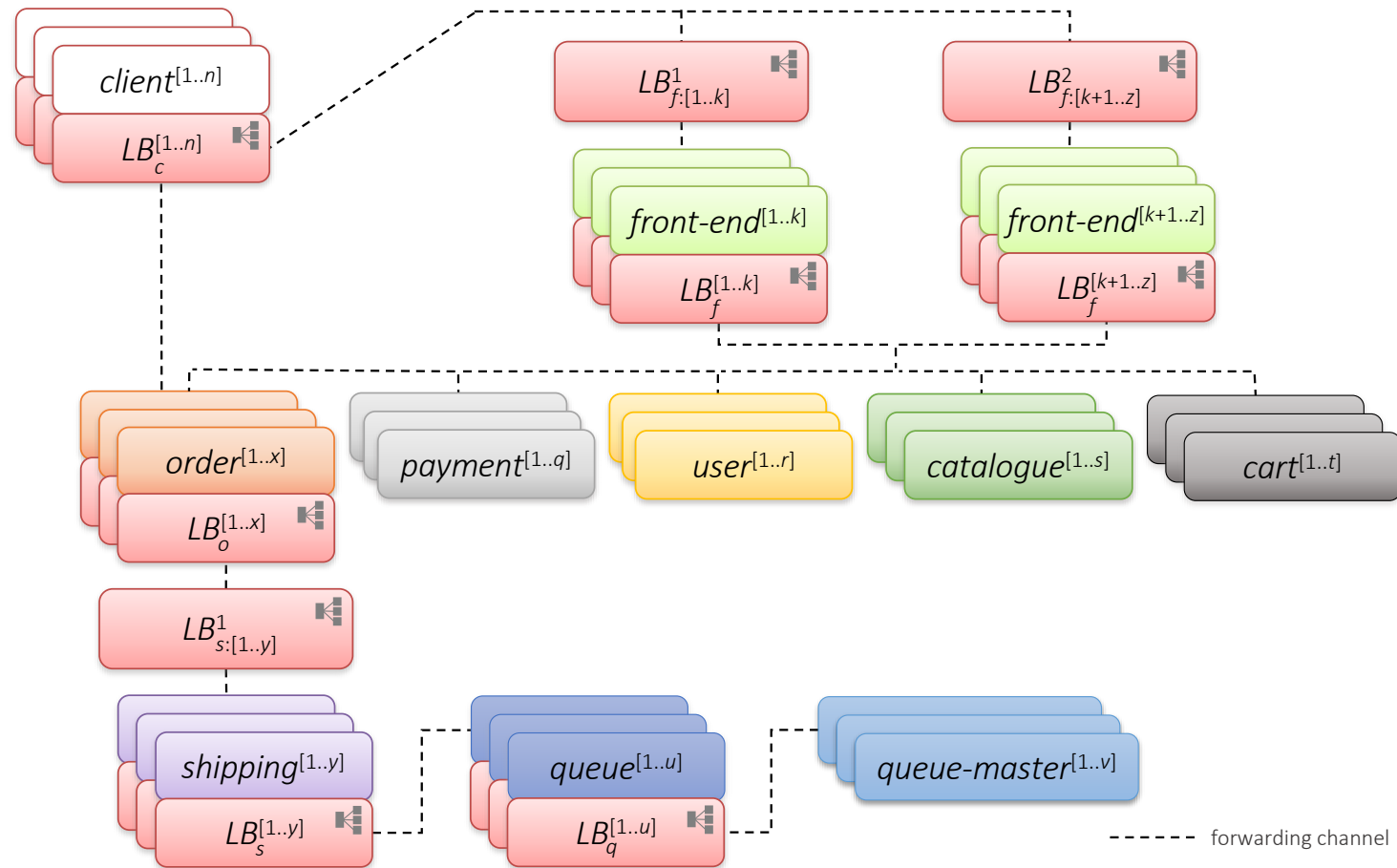
Includes benefits from both approaches

- + Improved fault tolerance
- + Better scalable approach
- + Enables *z-axis scaling*
- + Improved visibility on available instances
- + Internal structure hiding
- ! Still suffers latency due to extra-hop
- ! Still require propagation time
- ! It is not an always-valid substitutive alternative

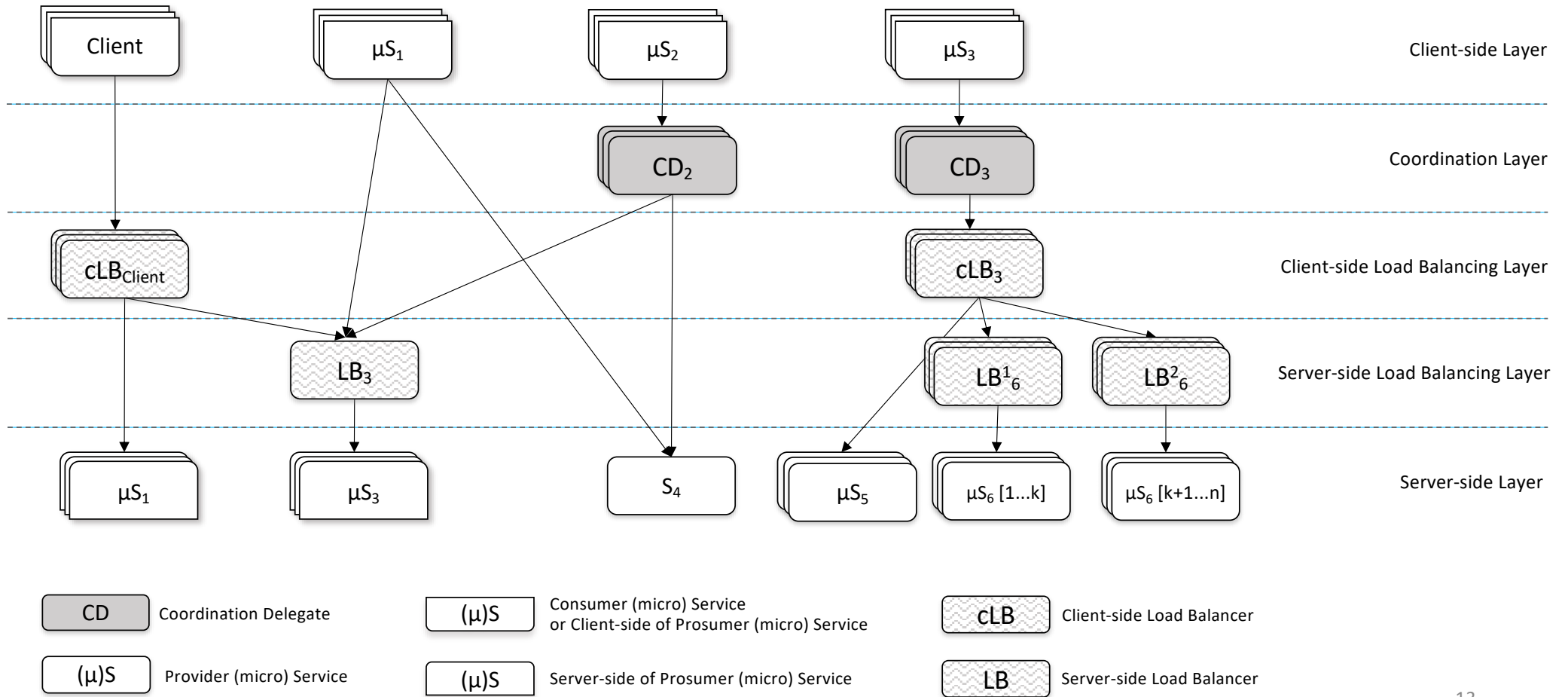
Hybrid approach at work: Sock Shop



Hybrid approach at work: Sock Shop

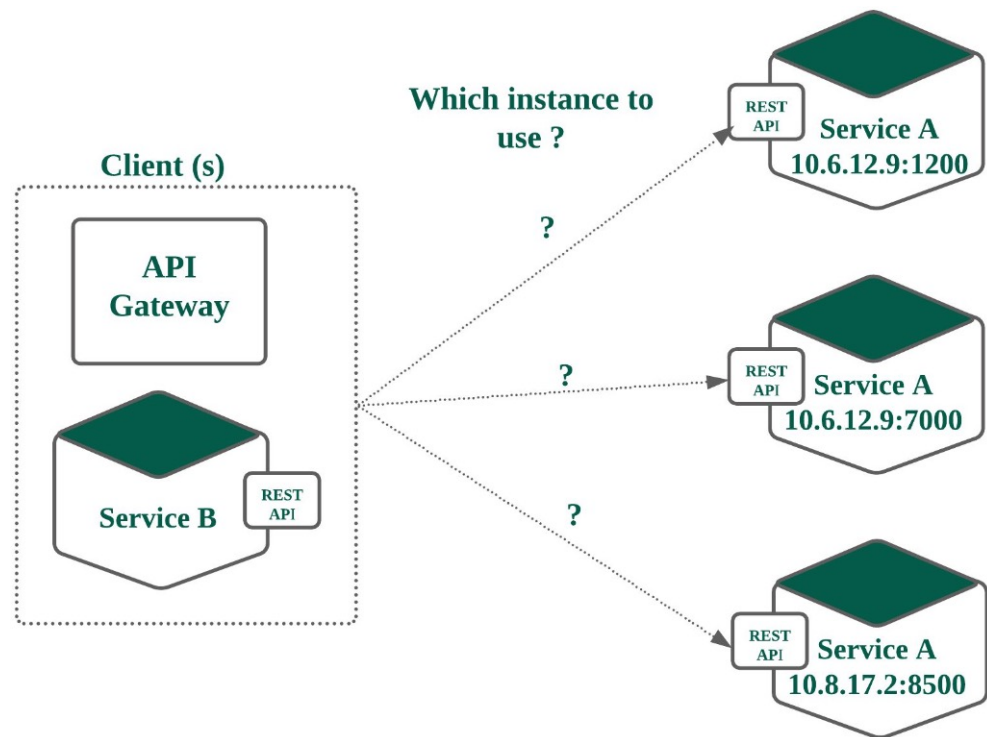


A layered architecture view



Service discovery

- › Load balancing does not happen for magic
 - › Extra architecture is needed
- › Clients and load balancers need to know the available instances
 - › Instances can appear and disappear dynamically
 - › Need to discover microservices instances
- › Use a service registry!

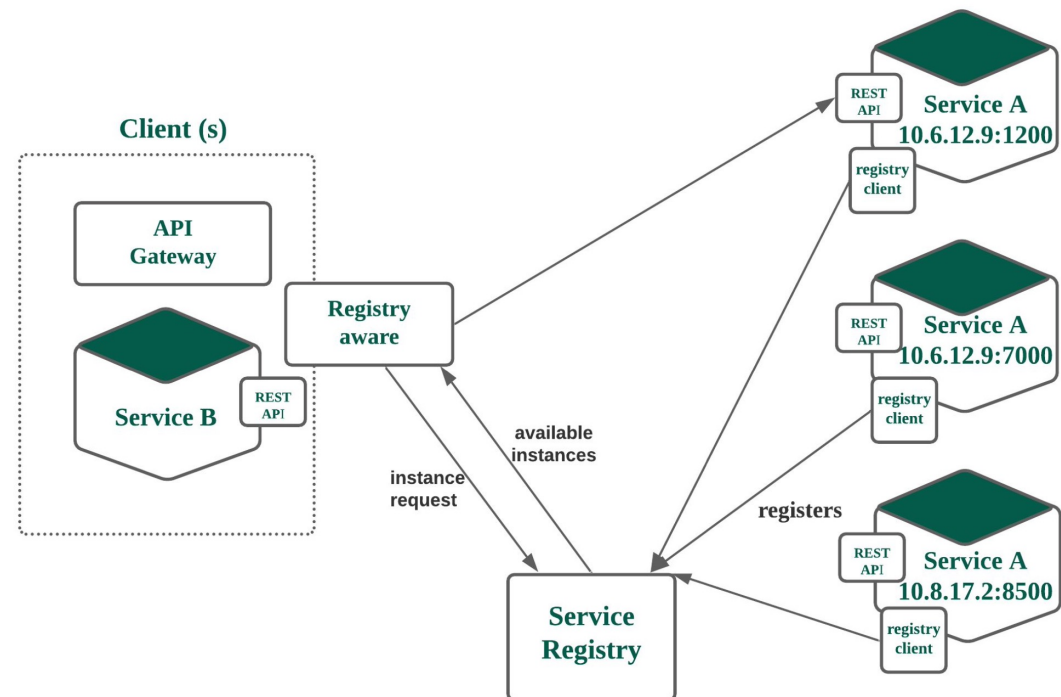


Client-side service discovery

...when clients perform load balancing

1. Microservices register themselves to the registry
2. Clients ask the available instances to the registry
 - › ...and select the instance to be invoked

- + Advantages from client-side LB
- Coupling between clients and service registry

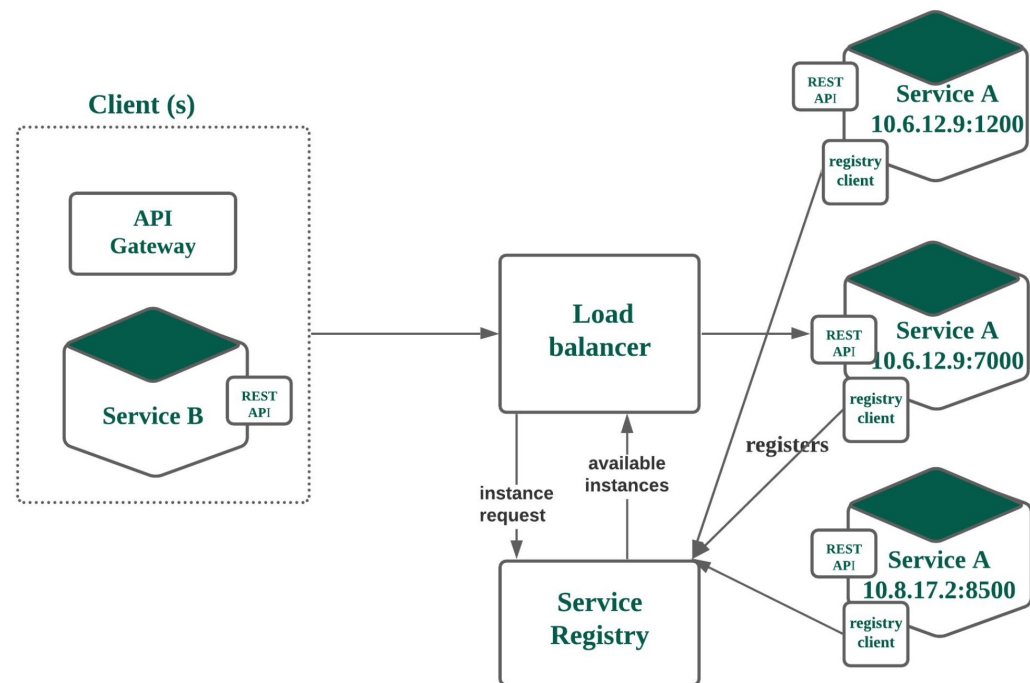


Server-side service discovery

› When there is client-side load balancing

1. Microservices register themselves to the registry
2. Clients send their messages to the server-side load balancer
3. The load balancer asks the available instances to the registry
 - › ...selects the instance and forwards the message

- + Decoupled from client(s)
- Drawbacks of server-side LB



Conclusions & Takeaways

Scalability is a key characteristic of Microservice architectures

- > Improve performances and fault-tolerance

Load Balancing enable to distribute requests among the (horizontally-scaled) services

Service discovery is required to keep track of the available microservice instances

Different Load Balancing approaches have different Pros and Cons

Hybrid Load Balancing mixes the benefits of client-side and server-side approaches

- > Alternative to pure centralized or distributed approaches

- > There is always a tradeoff

- > "It is better?" "It depends"

Conclusions & Takeaways

